

Checking the edge with an independent oracle

9 October 2026 | C003a boundary verification | Synthetic teaching examples

Sidiq review status: Not yet reviewed.

Research question and hypothesis

Does yesterday's C++ whole-year parser agree with a separately implemented reference at zero, the largest representable int, its immediate neighbours and malformed complete inputs? Hypothesis: exact value and exception-category comparisons will agree, including at the inclusive upper boundary. A useful verifier must also detect deliberately broken implementations.

Today teaches one concept: independent boundary checking. This completes the original October 8 slot on October 9. Yesterday implemented the parser; the fractional-truncation failure lesson, reproduction and consolidation follow in order. No extra lesson is added to catch up.

Beginner explanation and terms

A boundary is where the allowed outcome changes. An oracle supplies an expected answer using a separate method. Differential testing compares two implementations on identical inputs. A negative control deliberately breaks a rule to check that the verifier notices. Agreement is evidence for the tested cases, not proof for all possible inputs or evidence of personal mastery.

Our contract accepts nonempty ASCII decimal digits with numerical value from zero through M, where M is the target C++ int maximum. Leading zeros are allowed. Other characters are spelling errors, even when a numeric prefix is too large. Digit-only overflow is a range error. The two errors must remain distinct from the legitimate value zero.

Mathematics: test both sides of the boundary

For decimal digits d_1 through d_k , their value is the sum of $d_j \cdot 10^{(k-j)}$. Accepted text has $0 \leq \text{value} \leq M$. We test $M-1$, M , $M+1$ and $M+2$, with and without leading zeros. A strict inequality $\text{value} < M$ would wrongly reject M ; checking only $M-1$ would miss that mistake.

The probe reports $M = 2147483647$ on this host. Python forms $M+1 = 2147483648$ exactly. We never compute the overflowing addition inside a C++ int. The limit is queried from the compiled target, not inferred from Python's platform size or hard-coded as an assumption about all C++ systems.

Complete text	Expected and observed result	Reason
0	ok 0	Lower boundary is included
2147483646	ok 2147483646	Immediately below M
2147483647	ok 2147483647	Upper boundary is included
2147483648	range	First value above M
002	ok 2	Leading zeros preserve decimal value
2x	invalid	Suffix violates the spelling rule

The independent reference, step by step

The reference in `scripts/years_boundary_oracle.py` uses a full byte-pattern match followed by Python's integer conversion and comparison. C++ uses a character loop and `std::from_chars`. They share the written contract, but not the conversion implementation or bounded arithmetic.

1. `re.fullmatch(rb"[0-9]+", text)` asks whether every byte belongs to a nonempty sequence of ASCII digits. The `r` keeps the pattern literal and `b` makes it bytes. The plus means one or more; `fullmatch` prevents an accepted prefix from hiding a trailing newline or other forbidden byte.
2. If the match is absent, return the expected category invalid. This runs before conversion, so 1000 nines followed by `x` remain a spelling error. The oracle does not trim or decode the input.
3. `value = int(text, 10)` obtains the decimal integer. Python can represent values beyond the C++ int maximum, so it can calculate an expected answer on the other side of that boundary. Calling `int` alone would be wrong: its accepted grammar also allows signs, whitespace and underscores.
4. If `value > maximum`, expect range; otherwise expect ok followed by the exact decimal value. No floating-point approximation or tolerance is involved. An accepted but incorrect integer fails.

Preserving the original input in the C++ probe

The new `years_probe.cpp` is test plumbing, not the complete savings CLI. With `--int-max` it prints `std::numeric_limits<int>::max()`. Otherwise it reads all standard-input bytes into one `std::string` and calls the existing parser. Python sends the bytes directly through a pipe, without a shell.

The string constructor receives a beginning stream iterator and an end iterator. An iterator is a way to visit a sequence; here it copies bytes until end of input. It does not skip whitespace or stop at NUL. This preserves the exact contract input, including 2 followed by NUL followed by `x`. The iterator syntax is supporting test machinery; implementing general stream processing is deferred.

`argc` counts command arguments and `argv` holds their text. The test-only limit option is handled before input. Unexpected arguments or a detected stream error return exit status 2. A normal parser return prints ok and the value; catch blocks print invalid or range for the two expected exceptions. Other failures remain failures, rather than being reported as valid contract rejections.

The Python runner checks process success, exact output including its newline, empty error output and a ten-second timeout per invocation. A category is a test result, not a command failure; a mismatch raises an exception and makes the Make check fail. Each input uses a fresh process.

Fixed evaluation protocol

Synthetic strings have no market source or historical period. Cases are specified before running: integers 0 through 20; `M-1`, `M`, `M+1`, `M+2` and `10*M`; all 26 again with three leading zeros; then 23 malformed or long inputs. Total: 75. No random sampling, fitting or parameter search. Long inputs include 1000 leading zeros then 2, 1000 nines and those nines followed by `x`.

Implementation, checks and results

Added the C++ byte-preserving probe, Python oracle and cpp-years-oracle Make target; cpp-check now includes it. Existing parser and savings calculations are unchanged. Two direct oracle runs produce identical JSON. The archived result records this host's limit; other targets query their own.

Check	Observed result
Independent comparisons	75 agree: 47 accepted, 21 spelling errors, 7 range errors
Adjacent boundary	M accepted; M+1 and M+2 rejected as range errors
Wrong exclusive upper bound	Scratch mutation rejected at M
Wrong NUL truncation	Scratch mutation rejected for the three bytes 2, NUL, x
Full make check	608 Python tests; 66 savings and 30 parser checks; four output fixtures pass
Python quality	Ruff lint and 50 formatted files; strict mypy on 21 source files pass
Build and dependencies	Seven C++17 executables compile warning-clean; pip consistency passes

Both negative controls modify scratch copies only. One rejects the exact maximum; the other reconstructs input as a NUL-terminated C string, discarding the suffix and accepting 2. The verifier fails for the intended mismatch in each case. This establishes sensitivity to two faults; it is not a mutation-score or line/branch-coverage measurement. Implementation checks passed first run.

Reproduce from the repository root: make cpp-years-oracle; make check; .venv/bin/python scripts/years_boundary_oracle.py; .venv/bin/python -m pip --no-cache-dir check. The deterministic host result is in experiments/years_parser/boundary-oracle.json. Scratch logs and control sources are in work/2026-10-09. Render with make note NOTE_DATE=2026-10-09.

Worked financial connection and costs

Parsing 002 returns two whole model years. At principal $P = 1000$ units and annual rate $r = 0.05$, $B = P * (1+r)^2 = 1102.50$. Hand calculation: $1000 + 50 = 1050$, then $1050 + 52.50 = 1102.50$. The existing C++ demo still reproduces this output. Today verifies input mechanics, not returns.

Savings assumes a fixed effective annual rate, reinvestment and no fees, taxes, inflation or cash flows. Those omitted costs can reduce wealth. Passing M means only that the integer fits; it does not establish a sensible horizon or a finite compounded balance. C002 retains its financial checks.

Limitations and failure cases

The oracle trusts the compiled probe's reported target limit and standard-library behaviour. Both implementations could share a mistaken contract. Finite fixtures are not exhaustive; no security, performance or empirical-profitability claim follows. No licensed market data is used.

Python integer arithmetic is not resource-free. Decimal conversion has a configurable digit limit; these fixtures stay at 1000 significant digits, within the tested interpreter's default allowance. We do not disable that protection. The parser itself has no length cap; the local probe is not a hardened external interface. Current host and CI targets use ASCII-compatible execution encodings.

Primary sources consulted on 9 October

Python's numeric-types reference documents arbitrary-precision integer arithmetic. Its int reference describes a grammar broader than our policy and conversion-length protection. These support the reference method; the project's digit-only contract remains a deliberate local choice.

<https://docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex>

<https://docs.python.org/3/library/functions.html#int>

Microsoft's numeric_limits reference documents max() as the largest representable value. We use the target's reported limit and form neighbouring values in Python.

<https://learn.microsoft.com/en-us/cpp/standard-library/numeric-limits-class?view=msvc-170>

Interview explanation

I tested neighbouring boundary values using a separate arithmetic implementation, enforcing the same spelling policy and preserving original bytes. All 75 exact comparisons agreed, and two broken copies failed. This is bounded engineering evidence, not proof for all inputs or a financial forecast.

Progress, publication cadence and next task

The boundary-oracle slot is complete on October 9. C003a remains in progress. Next is the deliberate fractional-truncation failure lesson on October 10, followed by reproduction and consolidation. The original slot dates remain visible. C003b, vectors/statistics and the October 12 ledger stay conditional on prerequisites; do not compress lessons to meet those calendar headings.

Today's shared London-date draw is 1, with no earlier programme commits or pending unit. Prepare one coherent commit; record its hash, count and verified push in memory and the briefing. Publication authorisation does not change personal review status.

No input is needed for synthetic continuation. Q004a lawful access/sample evidence and Q005b integration still gate empirical research. Keep the 08:00 Europe/London schedule through December 31, including daylight-saving changes; then stop new research and request the next programme scope.

Check your understanding

1. Why test M as well as M-1 and M+1?
2. Why form M+1 in Python rather than adding one to a C++ int already holding M?
3. Why is plain Python int(text) an unsuitable reference for our complete contract?
4. What would go wrong if the test transport cut input at NUL?
5. Do 75 matching cases and two detected mutations prove correctness or profitable investing?

Suggested answers

1. The upper limit is inclusive. Testing only values below and above it can miss a mistaken rule that rejects the maximum itself. M-1 accepts, M accepts, and M+1 rejects.
2. Python can represent the larger integer exactly. The C++ addition exceeds that signed type's representable range, making the proposed boundary test itself invalid.
3. Python conversion allows spellings that our policy forbids, including surrounding whitespace, signs and underscores. First enforce a full ASCII-digit match, then compare the exact value with M.
4. The parser would receive different bytes from the intended test. The three-byte input 2, NUL, x could become the valid one-byte input 2, concealing the original spelling error.
5. No. They support the stated cases and two fault-detection claims. Untested cases, shared contract mistakes, resource constraints and financial-model limitations remain. No market return was measured.